

A Modular Testbed for Searching Single-UAV Foundation-Model Autonomy Architectures

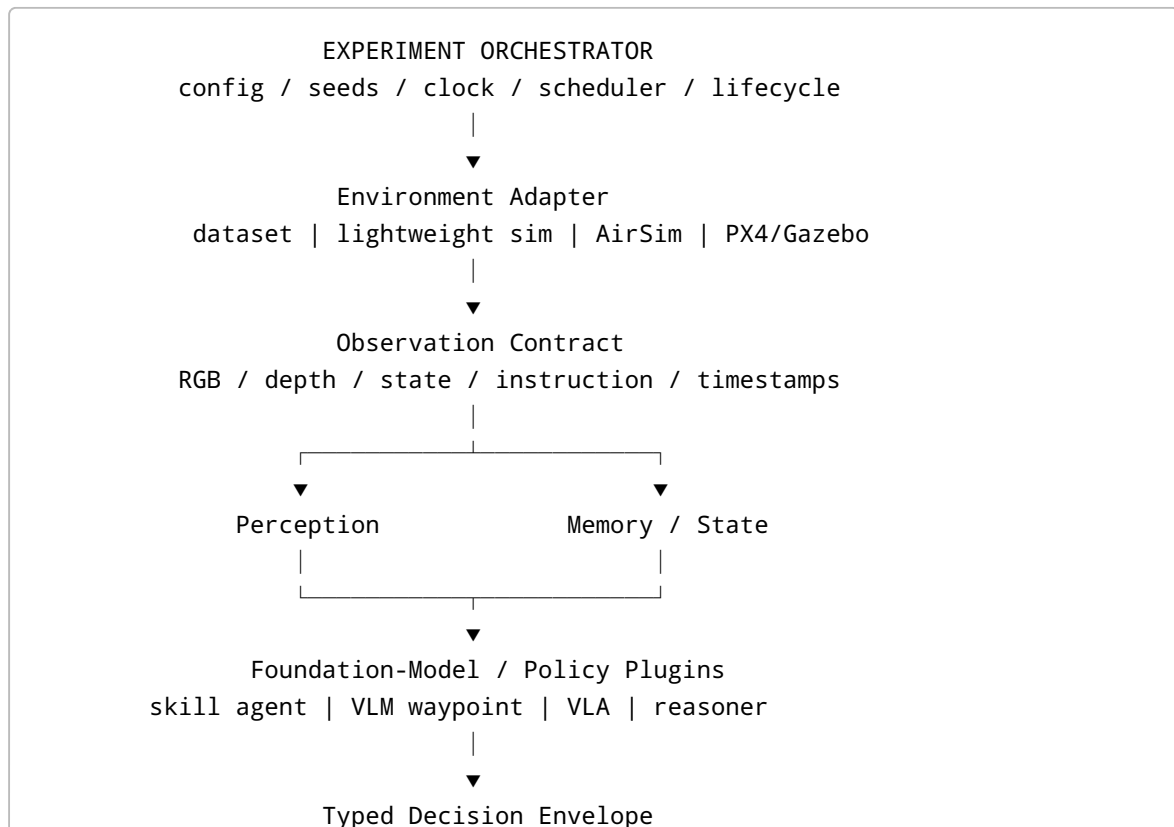
Executive summary

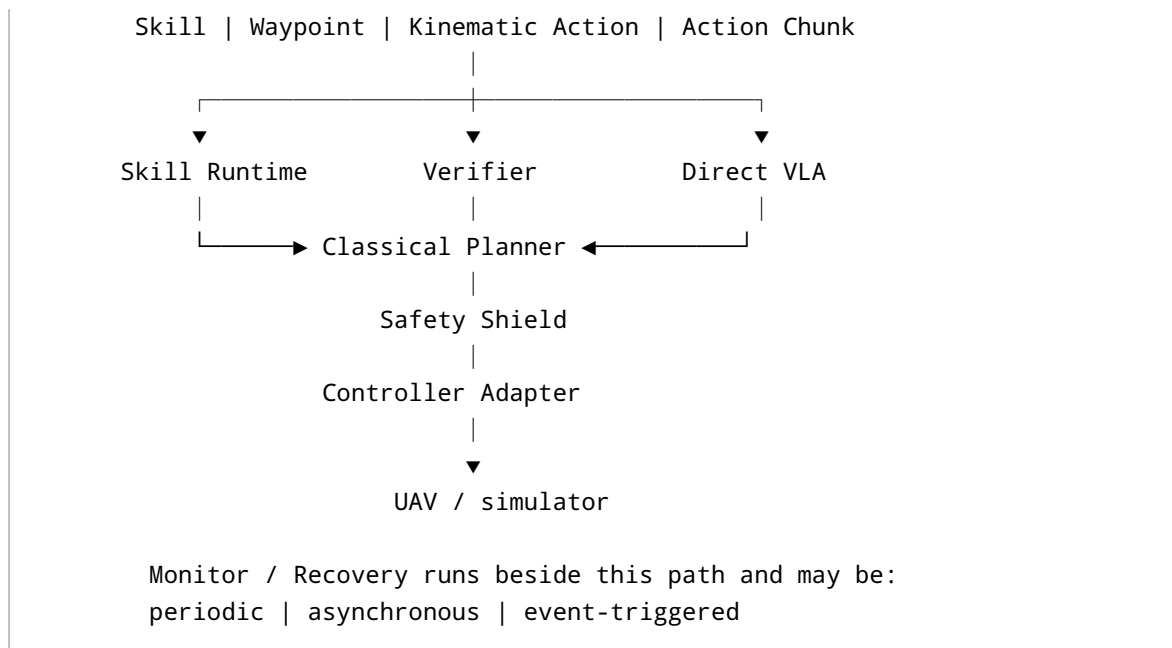
The most important design decision is **not** which UAV architecture to implement first. It is how to build the experimental substrate so that OnFly-like, AirHunt-like, CognitiveDrone-like, FLIGHT-like, skill-agent, waypoint, direct-VLA, and selective-recovery systems become **configurations of one runtime rather than separate codebases**.

The literature strongly supports this approach. Recent systems differ along a small number of consequential dimensions: where the foundation model hands off control, when expensive reasoning is invoked, how mission state is remembered, how progress/failure is monitored, how semantic outputs are prevented from becoming unsafe motion, and—inside learned-action systems—how much future motion is produced per inference. OnFly, for example, gets large gains from separating decision and monitoring loops and from retaining a classical planner; Selective Agentic Recovery shows that changing only the policy governing *when* reasoning is invoked can move hard-case success from 5% with local autonomy to 95% with learned selective invocation; CognitiveDrone-R1 and FLIGHT independently separate slower semantic reasoning from faster action execution. [filecite turn0file0](#)

1

The base system should therefore look like this:





The architecture study should **not** initially search every engineering optimization. Shared ViT features, exact KV-cache policy, TensorRT, AWQ, CUDA graphs, detector choice, model size, and keyframe-selection heuristics are important deployment parameters, but changing them during the first architecture screen would confound functional architecture with implementation efficiency. The prior design-space review reached the same conclusion and reduced the space to roughly five core architectural decisions plus one VLA-specific action-horizon decision. [filecite turn0file0](#)

I recommend implementing **eight sentinel configurations first** to prove the testbed is genuinely modular, while designing the configuration grammar to support the full **C0-C14 experimental set**. That allows Claude to build a useful repository before integrating every real model.

The generated design artifacts are ready to give Claude:

Modular UAV autonomy testbed architecture

[Download the main architecture as PNG](#)

[Download the main architecture as editable SVG](#)

[Download the experiment pipeline as PNG](#)

[Download the experiment pipeline as editable SVG](#)

[Download the full Claude-ready implementation prompt](#)

The central implementation principle is:

Architectures should be configuration; simulators, models, controllers, and datasets should be adapters.

That is deliberately different from reproducing AerialClaw wholesale. AerialClaw is valuable as an implementation reference because it explicitly separates an LLM “brain,” reusable skills, runtime validation, state/memory, and platform adapters. But adopting its LLM-centered control flow as the testbed itself would bias the experiment toward the `LLM → skills` family before the comparison even starts. ²

Literature-derived architecture fingerprints

Broad labels such as “hybrid VLM” or “VLA” are too coarse for this project. OnFly and AirHunt both combine semantic models with classical motion, yet OnFly creates explicit Decision and Monitoring roles with shared visual features, separate contexts/caches, compact historical memory and a verifier-plus-planner boundary, while AirHunt focuses on keeping continuous planning alive while expensive semantic reasoning is queried selectively. CognitiveDrone-R1 and FLIGHT both resemble “slow reasoner → fast learned executor,” yet they differ in state representation, concurrency and action horizon.

filecite turn0file0 3

NR below means that the requested property was not stated clearly enough in the primary material reviewed to treat it as an experimental fact.

System	FM role and inputs	Output / authority	Timing / invocation	Memory / perception sharing	Monitoring / recovery	Safety / cl execution
TypeFly	LLM interprets language mission and generates compact MiniSpec/ program representation	Skill/ program level	Streaming program interpretation rather than waiting for a verbose full plan	Program/ runtime state; no comparable long visual-memory architecture	Runtime handles execution progress	Drone pri bounded direct mo 4
AerialClaw	LLM receives mission, runtime context and feedback	Hard skills + reusable soft strategies	Closed-loop agentic reasoning	Document/ state memory and reflection	Runtime feedback/ reasoning loop	Capability and platfo bound wh can execu
PIVOT	General VLM observes images annotated with candidate actions/ points/ trajectories	Chooses from externally generated proposals	Iterative visual prompting	No UAV-specific long-horizon state	Repeated proposal refinement	Feasibility proposal q generic ro rather tha architectu
See, Point, Fly	VLM receives image + free-form instruction	Image-space 2-D waypoint + travel distance, then 3-D displacement	Repeated closed-loop VLM grounding	Primarily current observation	Implicit closed-loop replanning	Clean trai VLM → v archetype style dedi planner/n in its core 6

System	FM role and inputs	Output / authority	Timing / invocation	Memory / perception sharing	Monitoring / recovery	Safety / cl execution
FM-Planner	Benchmarks LLM/VLM methods and integrates language + vision for path planning	Global path-planning authority	Configuration dependent	Planner/world representation	Global replanning	Downstre navigation; classical; s primarily model-gu planning.
OpenFly-Agent	OpenVLA-derived model sees instruction, current image and selected history	Six discrete UAV action types	Repeated action generation	Keyframe-aware history + token merging	History supports long-horizon navigation	Physical s classical l trajectory and MPC; model inf external i experimen filecite: 8
CognitiveDrone	OpenVLA-derived VLA gets FPV imagery + language	Direct 4-D flight action	About 10 Hz in the reported implementation	No explicit long-horizon semantic-memory layer	No independent semantic monitor	Learned o with low-l stabilization the FM. filecite: 9
CognitiveDrone-R1	Adds a second VLM reasoner that interprets/ simplifies the task before the VLA	High-level reasoning delegated to direct-action VLA	Reasoner about 2 Hz, VLA about 10 Hz	Separate reasoning context; not an OnFly-style memory manager	Reasoner resolves semantic ambiguity	Same lear executor r classical p filecite: 9
GRaD-Nav++	CLIP/VLM semantic encoding + depth/state/ context feeds compact MoE policy	Body-rate + thrust, therefore very low-level learned authority	Policy ≈ 25 Hz; expensive semantic representation updated asynchronously every ten steps	Five-step history; sparse semantic refresh	Reactive learned policy	Depth ent for avoida independ local plan filecite: 10
AeroVLA / AerialVLA	Fused aerial views + language + coarse directional information	Continuous kinematic commands and landing	End-to-end learned action loop	No dedicated long-horizon monitor in core architecture	Reactive VLA	Low-level delegated controller minimalist stack. 11

System	FM role and inputs	Output / authority	Timing / invocation	Memory / perception sharing	Monitoring / recovery	Safety / cl execution
UAV-Flow	Benchmark/ training setup for instruction- conditioned fine-grained UAV control	Fine-grained learned flight actions	Short-horizon reactive setting	Not a new memory architecture	Task dependent	Useful pri controlled that learn policies ca outperform approach control re
DroneVLA	VLA semantically interprets manipulation mission	Prioritized task queue rather than direct flight action	Mission-level sequencing	Task-queue state	Task progression through queue	Grounding dynamic A estimation serving a specialize manipulat do most e work. filecite
AirHunt	VLM provides open-set semantic information during aerial object search	Semantic search guidance into continuous planner	Dual-path asynchronous execution; redundant semantic queries suppressed	Not a directly comparable OnFly-style KV/keyframe memory contribution	Semantic reasoning invoked according to need rather than dedicated mission monitor	Semantic- continuou combines informatio motion ef
OnFly	Same VLM performs local Decision and long-horizon Monitoring over shared visual features	Decision Agent proposes image-space goal; Monitor emits CONTINUE/ STOP/LOST	Fast decision loop + slower asynchronous monitor	first frame + selected keyframes + latest ; shared visual encoder/ features, separate agent contexts and KV caches	Explicit monitor; LOST induces stop/ reorientation	Semantic- waypoint followed b receding-l planner. filecite 14
Selective Agentic Recovery / PMR	Agentic reasoner is absent from nominal local execution and called for recovery	Chooses from bounded recovery actions/skills	Event-triggered using learned value-of- invocation	Compact runtime state plus mission summary	Recovery is the central semantic role	Parse → v shield → f local exec reasoner unrestrict authority.

System	FM role and inputs	Output / authority	Timing / invocation	Memory / perception sharing	Monitoring / recovery	Safety / cl execution
FLIGHT	Slow Streaming Pilot reasons about long-horizon semantic state while learned action model executes	Semantic state/ subgoal → continuous future action segments	Explicit asynchronous slow/fast architecture	Historical Pilot Reasoning state	Long-horizon semantic supervision	Fast learn action exe than OnFL classical l ¹⁶
ScoutVLA	Multimodal expert reasons about missing visual evidence; separate action expert executes	Continuous 5-DoF viewpoint-refinement trajectory	Decoupled experts	Local evidence-acquisition context rather than general long-horizon mission memory	Active perception determines next evidence acquisition	Flow-mate learned a no definin planner. 
WorldFly	Action branch coupled to future visual-state prediction	Direct learned action informed by imagined future	Coupled learned world/ action prediction	Explicit predictive future state	Implicit through prediction	No classic defining c useful as predictive architecture

The second half of the fingerprint matters just as much for reproducibility:

System	Action granularity	Compute / hardware	Representative latency or architecture evidence	Representative reported result	Verified code/ project route
TypeFly	Skill/ program	Laptop/drone runtime configuration varies	Streaming reduces time spent waiting for full generated programs	Up to 62% response-time reduction from compact/streaming design. ⁴	TypeFly GitHub
AerialClaw	Skill call	Designed for multiple UAV backends	Runtime validation separates reasoning from flight execution	Primary contribution is framework modularity rather than one navigation SR. ²	Paper is open-source framework; use source linked by primary paper

System	Action granularity	Compute / hardware	Representative latency or architecture evidence	Representative reported result	Verified code/ project route
PIVOT	Candidate action/ location	General VLM infrastructure	Multiple VLM iterations can be costly	Generic zero-shot robotics gains; not UAV-specific benchmark. ⁵	PIVOT project
See, Point, Fly	Semantic waypoint	Training-free VLM stack	Closed-loop VLM latency model dependent	Strong zero-shot aerial navigation and large gain over prior methods reported. ⁶	See, Point, Fly GitHub
FM-Planner	Global path	Simulation + real-world configurations	Model dependent	Compares eight LLM/ VLM planning approaches under standardized scenarios. ⁷	FM-Planner GitHub ; repository reviewed as still indicating fuller code release after review. ¹⁸
OpenFly-Agent	Discrete UAV actions	Jetson Xavier NX onboard for vehicle stack; reported FM inference external	History processing materially affects performance	Keyframe/history design gives a large gain over plain OpenVLA baseline. ⁸	OpenFly paper/ toolchain. ¹⁹
CognitiveDrone	Direct 4-D action	Reported ≈ 10 GB model-side footprint	≈ 10 Hz action loop	59.6% overall success on CognitiveDroneBench. ⁹	CognitiveDrone project
CognitiveDrone-R1	Reasoning + direct action	Two 7B models; reported ≈ 20 GB memory	Reasoner ≈ 2 Hz, VLA ≈ 10 Hz	77.2% vs 59.6% base; not a pure scheduling ablation because a second reasoning model is added. ¹⁰	CognitiveDrone project
GRaD-Nav++	Body rates + thrust	Jetson Orin Nano, ≈ 6 GB reported	Ordinary steps ≈ 30 ms; CLIP-refresh steps > 120 ms	Demonstrates fully onboard high-rate learned control with sparse expensive semantic refresh. ¹¹	Project page ; GRaD code

System	Action granularity	Compute / hardware	Representative latency or architecture evidence	Representative reported result	Verified code/ project route
AeroVLA	Kinematic command	Training/ runtime details in released project	End-to-end learned inference	Reports nearly 3× unseen-environment success over leading TravelUAV baselines. ¹¹	AeroVLA GitHub ; current repository reflects the renamed AeroVLA release. ²⁰
UAV-Flow	Fine-grained action	Benchmark-dependent	Reactive regime	VLA approaches outperform VLN baselines in the paper’s fine-control setting. ¹²	Primary paper/ dataset release
DroneVLA	Task queue	Task-specific system	High-level task sequencing	Architecture is important as evidence that “VLA” does not necessarily mean direct action. filecite turns off file0	Repository NR in the primary material verified here
AirHunt	Semantic planner guidance	Real-world aerial object-navigation stack	Asynchronous semantic querying	Primary paper reports improved efficient object-navigation through active semantic reasoning and continuous planning. ¹³	Primary paper states code/ data release intent; do not make the initial testbed depend on it
OnFly	Waypoint + monitor status	Jetson Orin NX 16 GB, fully onboard	Optimized Decision ≈ 0.81 s; Monitor ≈ 1.98 s, hybrid-memory monitor ≈ 1.15 s in the reported deployment	SR 67.8%; dual-agent removal 48.6%; planner removal raises collision 2.7→37.5%. filecite turns off file0	OnFly GitHub ; repository reviewed as not yet a dependency-ready implementation. ²¹
PMR	Recovery skill/ subgoal	Remote reasoner in reported experiments	Mean reasoner call ≈ 4.69 s, demonstrating why invocation policy matters	Hard/ambiguous cases: local 5%, periodic 62.5%, one-shot 75%, learned selective 95%. filecite turns off file0	Primary paper implementation details. ¹⁵

System	Action granularity	Compute / hardware	Representative latency or architecture evidence	Representative reported result	Verified code/ project route
FLIGHT	Continuous action chunk	Slow VLM + fast learned executor	Explicit asynchronous execution	Long-Horizon Flow reports substantially higher long-horizon completion than compared VLA/VLN baselines. ²²	FLIGHT project
ScoutVLA	Continuous trajectory/ chunk	Dual-expert VLA	Fast action expert separated from multimodal reasoning	Designed for active evidence acquisition rather than general long-range VLN. ²²	Repository NR in verified material
WorldFly	Direct action + future state	Learned dual branch	Joint action/ world prediction	Gains are targeted at partial observability, occlusion and viewpoint-change conditions. ¹⁷	Primary paper

This comparison exposes an important experimental issue: **published success rates cannot be directly used to rank architecture families**. SPF and OnFly are essentially zero-shot semantic-navigation systems; CognitiveDrone and AeroVLA train learned action policies; GRaD-Nav++ uses a specialized training pipeline; OpenFly trains on a large aerial VLN corpus. A raw leaderboard would therefore mix architecture, data, training regime, model size, action abstraction and benchmark difficulty. The common testbed has to remove as many of those confounders as possible.

FM-Planner is the closest existing precedent to your comparison idea because it evaluates multiple foundation-model planners under standardized drone path-planning conditions. Its scope, however, is primarily **global path planning**. The aerial-VLN surveys similarly classify end-to-end, hierarchical and related families while continuing to identify standardized benchmarking, long-horizon execution, continuous control and onboard deployment as open challenges. A factorized runtime in which the same modules are substituted under one flight substrate is therefore meaningfully different from reproducing systems paper by paper. ²³

Reduced design space and candidate architectures

The literature supports **six variables worth representing in the architecture grammar**, but only five should be crossed across the whole study. The sixth applies exclusively to learned-action executors.

Axis	Initial values	Why it survives reduction	Evidence / interpretation
Foundation-model authority	skill / semantic waypoint / learned kinematic action	Determines <i>where learned semantic intelligence hands off physical authority</i>	TypeFly/AerialClaw represent bounded skill authority; SPF/OnFly waypoint authority; CognitiveDrone/AeroVLA direct learned actions. Cross-paper results suggest task regime matters, but there is little clean matched-stack causal evidence—making this a core research axis. ²⁴
Reasoning schedule	periodic/ coupled / async multi-rate / event-triggered	Determines <i>when expensive semantic computation is allowed to affect flight</i>	OnFly dual-agent ablation materially changes SR; PMR holds most of the stack fixed while changing admission policy and shows a large event-triggering advantage; FLIGHT and GRaD-Nav++ separately use different-rate semantic/action computation. ²⁵
Safety/ execution boundary	planner/shield absent or present; semantic verifier optional only where meaningful	Separates semantic competence from physical safety	OnFly's planner removal is dramatically more harmful than removing only its verifier, so <code>planner/shield</code> is higher priority than testing dozens of verifier variants. ²⁵
Progress/ recovery architecture	implicit / periodic monitor / event-triggered recovery reasoner	Determines <i>who notices that the mission is no longer progressing</i>	OnFly gives mission progress an explicit asynchronous agent; PMR removes continuous semantic reasoning and invokes it only when recovery is valuable. ²⁶
Semantic memory	short context / compact task-relevant history	Determines <i>what mission history is available to semantic reasoning</i>	OnFly hybrid memory outperforms sliding-window memory; OpenFly's selected history/keyframe treatment substantially outperforms a plain OpenVLA history baseline. ⁸
VLA action horizon	one action / short action chunk	Conditional: only meaningful under learned action authority	FLIGHT and ScoutVLA separate reasoning from continuous multi-step learned action generation; recent VLA work such as DreamFly further reinforces receding-horizon action generation as a distinct design choice. ²⁷

Several tempting dimensions should **not** be first-stage architecture axes:

Fix initially	Reason
Model backbone/size within each modality	Otherwise “architecture” results become model-capability results. OnFly itself reports model-scale sensitivity. filecite turn0file0
Quantization, AWQ, TensorRT, CUDA graphs	Deployment optimization, not the functional location/timing of intelligence.
Shared vs duplicated visual encoder	Important after proving two semantic loops are worth having; OnFly makes it a strong later efficiency experiment. ¹⁴
Exact KV-cache implementation	Same reason: critical latency optimization, but not a first-order autonomy interface.
Twenty keyframe selectors	First ask <code>short context</code> vs <code>compact relevant history</code> ; tune selection later.
Detector architecture	Keep perception equivalent unless detector dependence is itself the question.
Body-rate/thrust vs kinematic VLA	It moves the learned/classical control boundary. GRaD-Nav++ and AeroVLA demonstrate both are feasible, but comparing them simultaneously with semantic architecture would introduce a major second problem. ²⁸
Diffusion vs autoregressive VLA implementation	Test action horizon first, not the entire action-model zoo.

For the first serious paper, I would use the following **15 configurations**, organized as controlled mutations rather than a Cartesian product. They correspond closely to the reduced architecture search developed in the uploaded design-space study. [filecite turn0file0](#)

ID	Configuration	Scientific question	Nearest comparison / motivation	Supporting result	Falsifying result
C0	Oracle semantic goal → fixed planner	How much failure is attributable to semantic architecture rather than flight execution?	Control ceiling	Very high SR establishes a competent substrate	Low SR means the planner/ controller/ environment stack must be fixed before FM conclusions
C1	LLM → predefined skills → planner	Is coarse, bounded mission authority enough?	TypeFly, AerialClaw	Competitive semantic success with low FM frequency	Large failure on visually grounded tasks

ID	Configuration	Scientific question	Nearest comparison / motivation	Supporting result	Falsifying result
C2	VLM → waypoint → fixed planner	Does visual semantic grounding justify waypoint-level authority?	SPF	Better grounding than C1 without large safety/latency penalty	No meaningful gain over skills
C3	C2 + semantic/geometric verifier	Does explicit waypoint validation help beyond the planner?	OnFly	Fewer invalid targets / modest SR gain	Negligible change; verifier can be fixed off later
C4	C3 + periodic low-rate progress monitor, short context	Does mission supervision matter independently of memory?	OnFly	Fewer overshoots, wrong stopping and lost-task cases	Same mission completion as C3
C5	C4 + compact keyframe memory	Does historical state specifically improve monitoring?	OnFly, OpenFly	Better long-horizon completion/stopping	No gain or unacceptable latency/memory cost
C6	C3 + event-triggered recovery reasoner, no persistent semantic monitor	Can reasoning disappear during nominal execution?	PMR	Similar/better hard-case SR with many fewer calls	Recovery success falls despite lower compute
C7	Direct VLA, one kinematic action, short context	Can learned visuomotor control eliminate waypoint planning?	AeroVLA, CognitiveDrone, UAV-Flow	Better fine-control/efficiency without unacceptable safety/generalization cost	Strong safety or semantic-generalization degradation
C8	C7 + same independent safety shield	Is direct VLA's safety weakness separable from its control capability?	PMR-style bounded execution, OnFly safety lesson	Collision reduction with little task penalty	Shield frequently modifies/rejects actions and destroys efficiency

ID	Configuration	Scientific question	Nearest comparison / motivation	Supporting result	Falsifying result
C9	C8 + compact history	Does direct VLA need explicit semantic history?	OpenFly	Better long-horizon task completion	No change despite extra compute
C10	Periodic slow reasoner → single-step VLA → shield	Does hierarchical semantic reasoning help direct VLA?	CognitiveDrone-R1	Gains concentrated on ambiguity/multi-stage tasks	Similar performance to C8
C11	C10 but chunked VLA executor	Can short future-action horizon bridge slow semantic inference?	FLIGHT, ScoutVLA	Smoother/high-frequency control with fewer semantic calls	More stale-action failures
C12	Async streaming reasoner → chunked VLA → shield	Is non-blocking fast/slow execution better than periodic hierarchy?	FLIGHT	Lower wall-clock delay/stalls with similar or better SR	Staleness offsets concurrency gains
C13	Event-triggered reasoner → best VLA executor → shield	Should high-level reasoning be invoked only upon uncertainty/no progress?	PMR + VLA hierarchy	Similar/better hard-case SR with sharply fewer calls	Trigger misses important semantic corrections
C14	Predictive world-model VLA sentinel	Does explicit future-state prediction justify added complexity under partial observability?	WorldFly	Targeted gain under occlusion/sharp viewpoint changes	No gain in precisely those failure regimes

The critical point is that this is a **conditional grammar**:

```
authority = skill | waypoint | direct_vla
```

```
if authority == "skill":
    executor = skill_runtime
    action_horizon = None
```

```
if authority == "waypoint":
```

```

planner = fixed_local_planner
verifier = off | semantic_geometric
action_horizon = None

if authority == "direct_vla":
    action_horizon = single | chunk
    safety_shield = off | on

semantic_supervision = (
    none
    | periodic_monitor
    | periodic_reasoner
    | async_reasoner
    | triggered_reasoner
)

semantic_memory = short_context | compact_history

```

Configurations such as `skill + diffusion action chunk`, `waypoint + no downstream conversion whatsoever`, or `monitor memory with no component that consumes the memory` should be **invalid configurations**, not additional experiments.

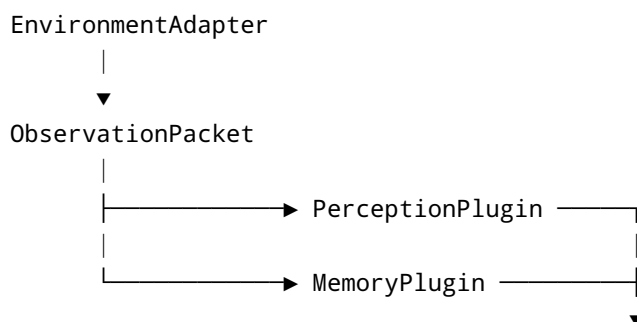
That single choice will save you from most of the combinatorial explosion.

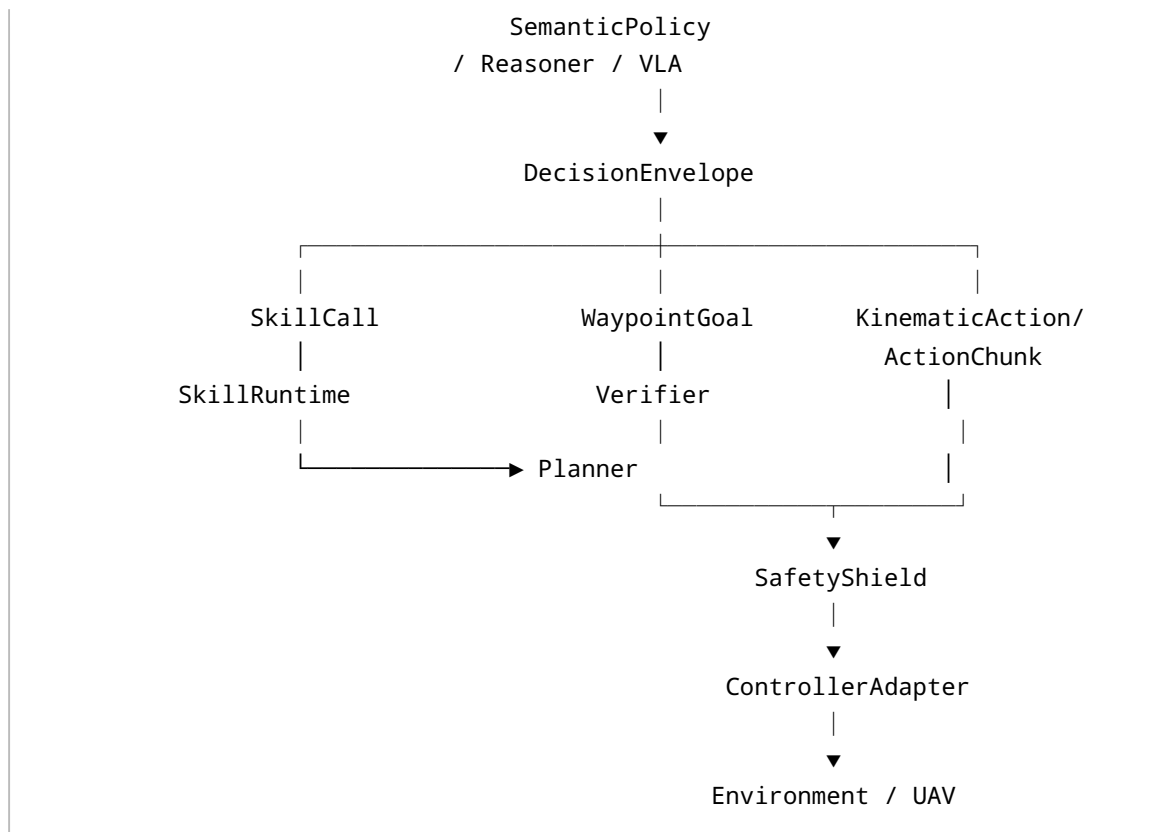
Recommended modular testbed architecture

The runtime should be a **thin typed kernel** with plugin interfaces around it. I recommend Python for the experimental core, `asyncio` for scheduling, Pydantic models for contracts, Hydra/OmegaConf for composition, and ROS 2 only at live robotics boundaries. ROS 2 provides explicit message/service/action interfaces and is a natural transport for PX4/Gazebo integration, while an in-process path is better suited to dataset replay and high-throughput simulation because it avoids forcing every offline experiment through robotics middleware. This division is a design recommendation based on the different use cases, not a claim that ROS 2 itself is unsuitable for experiments. ²⁹

The previous working plan already emphasized keeping observations, controller, safety conditions and evaluation comparable across architectures; the proposed runtime makes that requirement enforceable in software rather than relying on researcher discipline. [filecite](#) [turn0file1](#)

Core data path





The **DecisionEnvelope** is the most important contract in the system. Never let model-specific text or tensors flow directly to the controller. Every semantic/model output should be normalized into a small discriminated union:

```

DecisionEnvelope:
  kind:
    - skill
    - waypoint
    - kinematic_action
    - action_chunk
    - mission_directive

  payload: typed payload
  source_observation_seq: int
  source_t_sim_ns: int
  produced_t_wall_ns: int
  valid_until_t_sim_ns: int | None
  confidence: float | None
  provenance: dict
  
```

This solves several experimental problems simultaneously. It lets the same safety logger observe all architectures, lets you measure latency from observation to decision, lets you reject outputs generated from stale frames, and separates *semantic authority* from model-specific serialization. Measuring decision age is especially important in asynchronous systems: OnFly, AirHunt, FLIGHT and GRaD-Nav++ all permit slower semantic computation to coexist with continuing motion, so ordinary model latency

alone does not tell you how obsolete the state was when a decision was finally executed.

filecite turn0file0 30

The core contracts should be:

Contract	Minimum content	Why it is architecturally necessary
MissionSpec	mission ID, instruction, task family, success criteria, constraints	Same mission definition across architectures
ObservationPacket	RGB/depth refs, pose, velocity, optional IMU, camera intrinsics, sim/wall timestamps, sequence ID	Simulator independence and staleness measurement
PerceptionState	detections, optional reusable visual features, geometry, uncertainty	Allows raw VLM, detector+VLM and shared-feature variants
MemorySnapshot	bounded memory items, optional semantic summary, token budget	Separates semantic memory from planner's geometric map
DecisionEnvelope	typed semantic/action output plus provenance and timestamps	Universal architecture handoff
SafetyDecision	accept/modify/reject/stop, reason, risk	Comparable safety accounting
ProgressState	continue/stop/lost/blocked/ambiguous plus evidence	Supports OnFly- and PMR-like modes
RecoveryRequest	trigger, state summary, allowed recovery skills	Bounded event-triggered reasoning
Trajectory	time-indexed local trajectory	Common waypoint-planner handoff
ControlCommand	one canonical benchmark control representation, source decision, expiry	Controller independence
EpisodeEvent	component, event type, simulation/wall time, trace ID, payload	Unified logging

One subtle but important rule: **semantic memory and geometric planner state are different things.**

An ESDF/occupancy map used to avoid walls should not be counted as “the VLM has memory.”

Conversely, selected image/keyframe history should not become the flight controller's authoritative geometric world model. OnFly's architecture makes that distinction especially clear. 14

Another rule is to make scheduling a first-class plugin:

```
SchedulerPolicy =
  Periodic(rate_hz)
  | AsyncMultiRate(decision_hz, monitor_hz)
```

```
| EventTriggered(trigger, cooldown, max_calls)
| Hybrid(...)
```

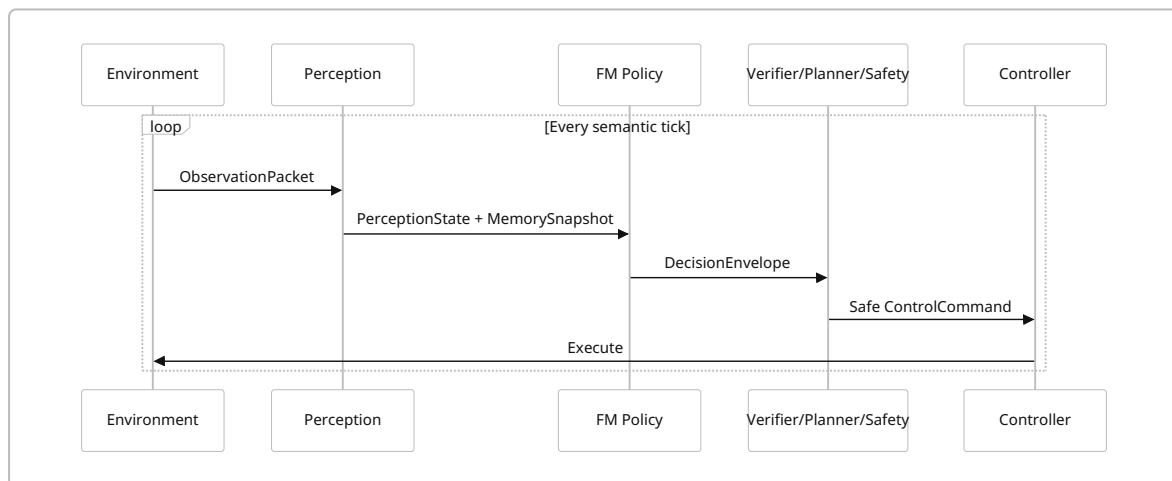
Do **not** let each policy implementation build its own private thread/timer logic. Otherwise C4, C6, C10, C12 and C13 will differ in both architectural timing and implementation quality.

The same applies to feature sharing. Create an optional:

```
FeatureCache[
  model_id,
  encoder_id,
  observation_seq,
  preprocessing_hash
] -> feature reference
```

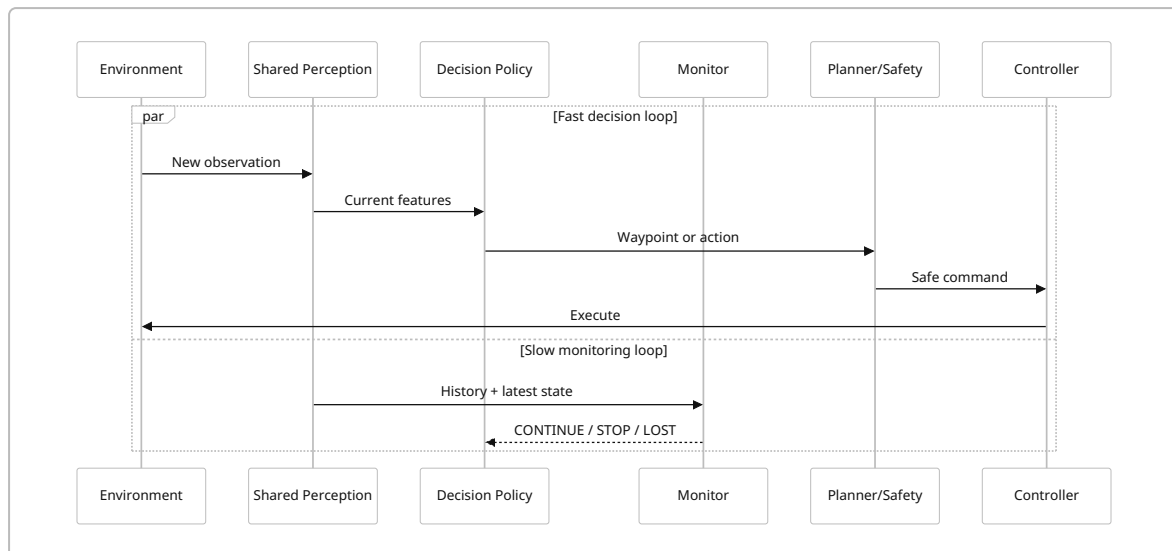
but keep it **disabled during the first causal architecture experiments**. Once a dual-loop architecture survives, enable shared visual features and cache reuse as a later compute experiment. This directly captures OnFly's useful engineering idea without hard-coding OnFly into the architecture. ¹⁴

Always-on semantic policy



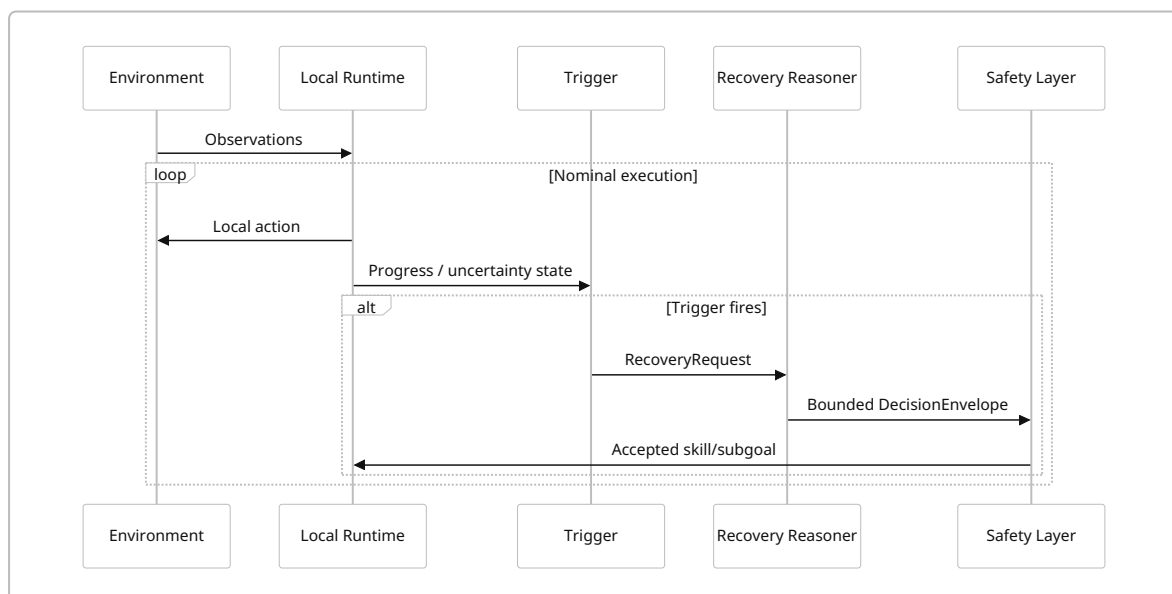
This covers C1–C3 and the simplest direct VLA configurations.

Multi-rate dual-loop mode



This supports an OnFly-like configuration, but the monitor could be a different model, the visual features need not be shared, and the decision policy need not produce waypoints. That distinction is exactly what makes this a benchmark runtime instead of an OnFly reproduction. ¹⁴

Event-triggered semantic recovery



This isolates PMR's most interesting idea—**reasoning admission**—from its particular mission runtime. The PMR paper is especially valuable experimentally because its comparisons hold much of the execution stack constant while altering the reasoning-invocation policy. ¹⁵

The complete architecture graphic and the independent experiment-pipeline graphic are downloadable here:

[Main testbed architecture — SVG](#)

[Experiment pipeline — SVG](#)

Reference stack, simulators, controllers, and reusable code

I would avoid choosing one heavyweight simulator as the architectural center. The **EnvironmentAdapter** should be authoritative; simulators are replaceable implementations.

The recommended stack is:

Layer	Initial recommendation	Why
Core language	Python 3.10/3.11 compatible code	Maximum compatibility with current VLM/VLA research repositories while keeping modern typing/async support
Contracts	Pydantic v2 + Python <code>Protocol</code> /ABC	Runtime validation and implementation-neutral plugin interfaces
Composition	Hydra + OmegaConf YAML	Architecture = configuration; inheritance for C0–C14
Scheduling	<code>asyncio</code> + injectable simulation clock	Supports periodic, multi-rate and triggered semantics without multiple runtimes
Fast CI environment	Minimal deterministic Gym-style 3-D environment	Architecture logic can be tested without CUDA, ROS or a photorealistic simulator
High-throughput optional sim	VisFly	Provides Gym-style UAV simulation built for large-scale learning/evaluation. ³¹
Dynamics/control validation	PX4 SITL + Gazebo	PX4 officially supports Gazebo simulation and ROS 2 integration, making it appropriate for controller/dynamics realism and later HIL. ³²
Vision-rich environment	Project AirSim	The original Microsoft AirSim repository is no longer the active development path and points users toward newer alternatives; Project AirSim provides a current UE5-based continuation with Python/network APIs. ³³
Aerial VLN environment	AerialVLN/AirVLN adapter	Existing aerial-language simulator with city-scale scenarios; valuable for instruction-following tasks. ³⁴
Real/PX4 command path	ROS 2 PX4 interface or MAVSDK Offboard adapter	MAVSDK exposes velocity/yaw Offboard setpoints while PX4 exposes broader ROS 2 integration. ³⁵
FM serving	Local Transformers or OpenAI-compatible local endpoint	Keeps VLM/LLM provider independent from architecture runtime
Deployment backend	ONNX/TensorRT/quantized variants as deployment profiles	Do not encode these as architectural choices

Layer	Initial recommendation	Why
Logs	Parquet + JSONL; optional rosbag at ROS boundary	Fast analysis plus complete human-readable event trace
Analysis	Polars/Pandas + DuckDB, bootstrap/ paired statistics	Cross-run and per-scenario analysis without an invented single score
CI	pytest + ruff + pyright/ mypy	Contract and timing regressions are dangerous in this project

For a new project I would **not** make legacy Microsoft AirSim the required base. Its official repository states that the old project will be archived/no longer receive updates, while Project AirSim is a current successor created by former AirSim engineers. Legacy AirSim compatibility can still be useful because several UAV datasets and papers use it, but it should be just another adapter. ³³

Likewise, I would not make PX4/ROS mandatory for the core unit tests. PX4/Gazebo is the right place to validate dynamics and recovery, but forcing every architecture sweep through SITL would make cheap architecture screening unnecessarily slow. The same configuration should first be executable against a deterministic in-process environment and later promoted to PX4/Gazebo. This is an architectural recommendation inferred from the role each platform serves. PX4's official interfaces make that adapter boundary practical. ³⁶

The most useful existing repositories are **component donors**, not foundations that should dictate the architecture:

Existing work	Reuse	Do not inherit
AerialClaw	Skill interface, capability boundary, mission runtime concepts, platform adapters	Its assumption that an LLM “brain” is the central organizing component. ²
TypeFly	Skill/program planner baseline	MiniSpec as the universal internal representation. ⁴
See, Point, Fly	Clean VLM-waypoint plugin	SPF-specific navigation loop as the global runtime. ³⁷
AeroVLA	Direct-VLA model adapter/checkpoint/training assets	Its kinematic-action representation as a universal interface for every architecture. ²⁰
OpenFly	Dataset/environment tooling, history/keyframe ideas	OpenVLA-specific architecture as the benchmark kernel. ³⁸
OnFly	Monitor contract, shared-feature experiment, compact-memory experiment, verifier interface	Dependency on its unreleased/incomplete repository implementation; reproduce the abstractions independently first. ³⁹

Existing work	Reuse	Do not inherit
FM-Planner	Scenario/evaluation methodology and global-planning baseline	Assuming it already provides a reusable architecture-comparison framework; current repository status should be checked before making it a dependency. ⁴⁰
PX4	Real controller, SITL/HIL, flight-command adapter	Flight-stack messages leaking into semantic model APIs. ⁴¹
Project AirSim	Vision-rich simulator adapter	Simulator-specific image/pose classes in the domain contracts. ⁴²

A good repository layout is therefore:

```

uav_arch_lab/
├── configs/
│   ├── architectures/
│   ├── environments/
│   ├── models/
│   └── experiments/
├── src/uavlab/
│   ├── contracts/
│   ├── core/
│   │   ├── orchestrator.py
│   │   ├── scheduler.py
│   │   ├── decision_router.py
│   │   ├── clock.py
│   │   └── registry.py
│   ├── plugins/
│   │   ├── perception/
│   │   ├── reasoning/
│   │   ├── memory/
│   │   ├── verifier/
│   │   ├── planning/
│   │   ├── control/
│   │   ├── monitoring/
│   │   ├── recovery/
│   │   ├── inference/
│   │   └── metrics/
│   ├── adapters/
│   │   ├── dataset_replay/
│   │   ├── gym/
│   │   ├── project_airsim/
│   │   └── gazebo/

```

```

|   |   | px4/
|   |   | ros2/
|   |   | └─ mavsdk/
|   |   |
|   |   └─ experiments/
|   |       └─ analysis/
|   |
|   └─ tests/
|       └─ unit/
|       └─ contract/
|       └─ integration/
|       └─ smoke/
|
└─ docs/
    └─ architecture.svg
    └─ mermaid/

```

A controller abstraction deserves special care. The runtime can support both local-velocity setpoints and delta-pose actions, but **each matched experiment must freeze one direct-action representation**. Do not compare a VLA that controls body rates and thrust with another architecture that receives collision-checked position targets and call that an architecture-only comparison. GRaD-Nav++ and AeroVLA illustrate how different those authority boundaries are. [filecite turn0file0](#) 28

Coordinate frames should likewise be explicit in every contract. Define a canonical internal frame, provide tested ENU↔NED conversions at adapters, and never allow a plugin to infer frame conventions from context.

Evaluation protocol and automated search

The test harness should treat **environment, architecture and episode** as separate objects:

```

ArchitectureConfig
  +
  EpisodeSpec
  (task, scene, seed, failure injection)
  +
  EnvironmentConfig
    |
    ▼
  Experiment Orchestrator
    |
    ▼
  Canonical Event Log
    |
    ▼
  Metrics + paired analysis

```

Controlled architecture experiment pipeline

Every run should persist a reproducibility manifest containing the source Git commit, dirty-tree status, architecture-config hash, model/checkpoint IDs, prompt hashes, simulator build, seeds, hardware, container/dependency versions and timing configuration.

The original short experiment plan proposed AerialVLN-style instruction following, semantic object/goal search, and CognitiveDrone-style ambiguity/recovery tests. That remains a good starting point, but I would add a fourth task regime: **fine-grained reactive maneuvering**. The literature is specialized enough that one architecture may dominate long-horizon semantic search while another dominates rapid control; UAV-Flow explicitly finds a VLA advantage in its fine-control regime, whereas OnFly/SPF focus on open-vocabulary semantic navigation. [filecite turn0file1](#) [43](#)

The benchmark should therefore contain:

Regime	What it tests	Useful source
Long-horizon language navigation	Semantic grounding, progress, stop correctness, memory	AerialVLN/OpenFly-style episodes. 44
Open-set object/goal search	Semantic exploration and target commitment	AirHunt/SPF/OnFly-style scenarios. 45
Failure/ambiguity/recovery	Blocked route, wrong target, no progress, stale decisions	PMR/CognitiveDrone-inspired failure variants. 46
Fine continuous maneuvering	Control authority, reaction frequency, action horizon	UAV-Flow, CognitiveDrone, GRaD-Nav++. 47
Optional partial observability sentinel	Occlusion/viewpoint-change sensitivity	WorldFly/DreamFly. 48

Do not collapse evaluation into one arbitrary score. The core outputs should form a **Pareto surface**:

Capability	Systems/compute property
Task success rate	median and p95 semantic-decision latency
Collision / safety-violation rate	actual control-update frequency
Correct mission termination	semantic decision age when executed
Recovery success	peak RAM/VRAM
Semantic/subgoal completion	reasoner calls / generated tokens
SPL/path efficiency and flight time	optional power/energy per successful mission
Executable-plan rate	safety rejection/modification rate
False-target commitment	stale-action rate

The **age of a semantic decision** should be a first-class metric:

```

decision_staleness =
    execution_sim_time
    -
    source_observation_sim_time

```

It is more informative than inference latency alone for asynchronous architectures. A slow output generated from an image that is still operationally valid is very different from an equally slow output produced while the UAV has already flown several meters away. This is precisely the kind of hidden architecture effect that appears when semantic reasoning and motion run at different rates.

filecite turn0file0

The experimental order should be staged:

Macro screening: C0, C1, C2 and C7 establish whether skill, waypoint or direct learned authority is worth pursuing.

Safety screening: C2→C3 and C7→C8 isolate semantic verification and independent safety. OnFly's planner ablation makes it especially important not to compare a shielded waypoint stack against an unshielded VLA and then attribute the difference to "modularity." filecite turn0file0

Timing and recovery: compare C3/C4/C6 on the waypoint side and C8/C10/C12/C13 on the learned-action side.

Memory and action horizon: compare C4→C5, C8→C9 and C10→C11.

Predictive state: run C14 only if occlusion/partial-observability failures are still substantial.

Deployment optimization: only on the two or three Pareto-winning stacks should you then investigate shared ViT features, KV caching, quantization, TensorRT, model scaling and other compute optimizations.

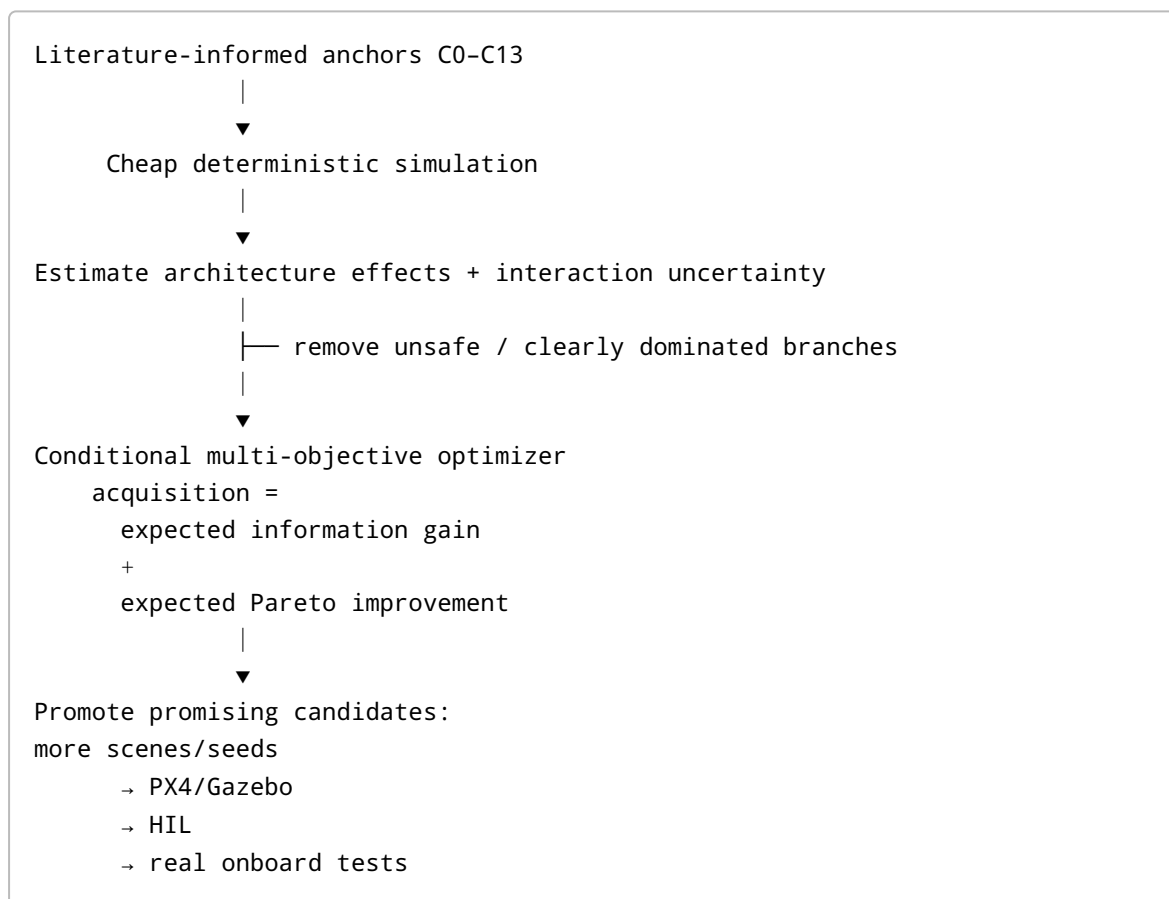
The highest-priority interactions are:

Interaction	Why it matters
Authority × safety boundary	A waypoint planner and a direct VLA expose different physical authority; OnFly shows the safety boundary alone can dominate collision results. filecite turn0file0
Reasoning schedule × recovery policy	PMR favors selective invocation in hard cases, while OnFly benefits from persistent low-rate monitoring. These are competing hypotheses, not interchangeable labels. 49
Memory × monitoring	Long-term history may matter far more for "am I still accomplishing the mission?" than for instantaneous steering. OnFly/OpenFly support this interpretation. filecite turn0file0
Action horizon × semantic refresh	Longer chunks can bridge slow reasoning but also make stale high-level intent persist longer. FLIGHT makes this interaction central. 16

Interaction	Why it matters
Compute cost × actual control frequency	GRaD-Nav++ sparsifies expensive CLIP updates while preserving a high policy rate; OnFly similarly separates slower semantic inference from faster motion planning. ⁵⁰
Async reasoning × decision staleness	Nonblocking inference only helps when late semantic outputs remain useful.
Dual semantic loops × shared perception/cache	A late efficiency study motivated strongly by OnFly; first prove the second loop itself is valuable. ¹⁴
Task horizon × authority	UAV-Flow's reactive setting and long-horizon VLN papers suggest there may be no globally dominant authority level. ⁵¹

For automated architecture search, I would **not begin with neural architecture search**. The first phase is experimental identification over a conditional systems grammar.

A practical algorithm is:



After the anchor experiments, a multi-fidelity Bayesian optimizer or evolutionary categorical optimizer can search only legal configurations. The objectives should remain vector-valued—success, safety, recovery, latency, memory and flight efficiency—rather than being combined into one arbitrary reward. AutoPilot provides earlier precedent for Bayesian UAV design-space exploration, although it searched cyber-physical hardware/autonomy co-design rather than foundation-model runtime architecture. ⁵²

The eventual scientific object is therefore not:

“the universally perfect UAV architecture”

but:

$$A^* = f(\text{task regime, semantic uncertainty, flight dynamics, compute budget, latency budget})$$

That formulation also makes your later RQ2 natural: the system could eventually select or reconfigure its own semantic-reasoning mode during flight.

Claude-ready implementation specification

The full implementation prompt—including repository structure, typed schemas, eight complete configuration examples, CI requirements, test scripts and Mermaid diagrams—is available here:

[Download](#) `claude_uav_testbed_prompt.md`

The following is the compact paste-ready specification I would place at the beginning of a Claude Code session.

You are building a research testbed for single-UAV autonomy architectures.

This is NOT an implementation of one paper.

Architectures must be CONFIGURATION, not code forks.

GOAL

Build one modular runtime capable of instantiating and fairly comparing:

- LLM -> predefined skills
- VLM -> semantic waypoint -> classical planner
- waypoint + verifier
- waypoint + asynchronous progress monitor
- waypoint + event-triggered recovery reasoner
- direct VLA
- direct VLA + independent safety shield
- slow reasoner -> fast VLA
- asynchronous fast/slow VLA
- event-triggered reasoner -> VLA
- optional world-model VLA

The same architecture config must be runnable against:

1. a deterministic lightweight CI simulator,
2. dataset replay,
3. Project AirSim,
4. PX4 SITL + Gazebo,
5. eventually real/HIL PX4.

Do not make ROS 2, PX4, AirSim, a particular VLM, or AerialClaw the core abstraction.

ARCHITECTURAL PRINCIPLE

EnvironmentAdapter

- > ObservationPacket
- > Perception + Memory
- > SemanticPolicy / VLA / Reasoner
- > DecisionEnvelope
- > Verifier / Planner / SafetyShield
- > ControllerAdapter
- > UAV

Monitor and Recovery run as independently schedulable semantic roles.

Use:

- Python
- asyncio
- Pydantic v2
- Protocol/ABC plugin interfaces
- Hydra/OmegaConf YAML configuration
- pytest
- Parquet + JSONL logging

ROS 2 belongs at the live robotics adapter boundary.

Offline replay and lightweight simulation should run in process.

REQUIRED CONTRACTS

MissionSpec

ObservationPacket

PerceptionState

MemorySnapshot

DecisionEnvelope

SafetyDecision

ProgressState

RecoveryRequest

Trajectory

ControlCommand

EpisodeEvent

DecisionEnvelope must be a discriminated union:

```
kind =  
    skill  
    | waypoint  
    | kinematic_action  
    | action_chunk  
    | mission_directive
```

Every DecisionEnvelope MUST include:

- decision_id
- source_observation_seq
- source_t_sim_ns
- produced_t_wall_ns
- valid_until_t_sim_ns
- confidence
- provenance

No free-form LLM/VLM text may ever reach a controller.

PLUGIN INTERFACES

EnvironmentAdapter
 PerceptionPlugin
 MemoryPlugin
 SemanticPolicy
 MonitorPlugin
 RecoveryPolicy
 VerifierPlugin
 PlannerPlugin
 SafetyShield
 ControllerAdapter
 InferenceBackend
 MetricsSink

SCHEDULING

Implement one Scheduler with interchangeable policies:

PeriodicPolicy(rate_hz)
 AsyncMultiRatePolicy(decision_hz, monitor_hz)
 EventTriggeredPolicy(trigger, cooldown_s, max_calls)
 HybridPolicy(...)

Scheduling must not be implemented privately inside model plugins.

Every semantic result must be checked for staleness before execution.

Log:

```
decision_age =
    execution_sim_time - source_observation_sim_time
```

DECISION ROUTER

skill:
 SkillRuntime
 -> optional subgoal
 -> verifier/planner
 -> safety
 -> controller

```

waypoint:
  verifier
  -> fixed local planner
  -> safety
  -> controller

kinematic_action:
  safety
  -> controller

action_chunk:
  chunk executor / receding horizon
  -> safety
  -> controller

mission_directive:
  scheduler/monitor state only
  NEVER direct motor authority.

```

MEMORY

Keep semantic mission memory separate from geometric planner state.

Provide:

- no_memory
- short_context
- compact_keyframe_memory
- compact_semantic_state

Do not initially implement 20 keyframe heuristics.

PERCEPTION SHARING

Implement an optional FeatureCache keyed by:

```

(model_id,
 encoder_id,
 observation_seq,
 preprocessing_hash)

```

Keep it OFF by default during the first architecture comparison.
 Enable it later to study shared-perception efficiency.

Multiple semantic agents may share visual features while retaining separate contexts/KV caches.

COORDINATE FRAMES

Define one canonical internal command convention.

Provide explicit, unit-tested ENU/NED transforms.
Never perform silent frame conversion.

For controlled direct-VLA comparisons, freeze one kinematic action representation for the whole matched experiment.

INITIAL EIGHT CONFIGURATIONS

C1:

LLM -> skills -> fixed planner

C2:

VLM -> waypoint -> fixed planner

C3:

C2 + semantic/geometric verifier

C5:

VLM waypoint decision loop
+ low-rate asynchronous progress monitor
+ compact keyframe memory
+ verifier
+ fixed local planner

C6:

C3

+ local progress detector
+ reasoner invoked ONLY on blocked/no-progress/ambiguity events

C7:

direct VLA -> one kinematic action
no planner
no shield

C8:

same exact C7 VLA
+ independent safety shield

C12:

asynchronous slow streaming reasoner
-> fast chunked VLA executor
-> independent safety shield

These must differ through YAML configuration, not copied runtimes.

CONFIG VALIDATION

Reject illegal combinations such as:

- action_chunk with authority=skill
- mission_directive sent to controller

- waypoint architecture with no downstream conversion unless explicit `unsafe_ablation=true`
- memory configured for a semantic component that does not exist
- monitor frequency configured without a monitor

FAKE PLUGINS FIRST

Before integrating real models implement:

- scripted skill policy
- oracle waypointer
- random waypointer
- mock VLA
- identity perception
- no memory
- FIFO memory
- compact keyframe memory
- no verifier
- bounds verifier
- simple collision shield
- straight-line/local planner
- mock controller
- deterministic 3-D Gym-style environment

The repository must run completely without CUDA, ROS, Gazebo, a network connection, or model downloads.

ENVIRONMENT ADAPTER PRIORITY

1. deterministic lightweight environment
2. PX4 SITL + Gazebo
3. Project AirSim
4. dataset replay adapters
5. optional VisFly/high-throughput backend

Do not make legacy Microsoft AirSim a required dependency.

LOGGING

Every module emits `EpisodeEvent`.

Persist:

- Parquet event table
- JSONL human-readable trace
- optional rosbag at ROS boundary

Every run stores:

- git SHA
- dirty-tree status
- architecture config hash
- model/checkpoint hashes

- prompt/template hashes
- simulator/build version
- random seeds
- hardware
- dependency lock hash
- container image digest when available

METRICS

Required:

- task success
- collision / safety violation
- correct terminal stop
- recovery success
- subgoal completion
- path efficiency / flight time
- median and p95 semantic latency
- actual control update frequency
- decision staleness
- executable-plan rate
- rejected/modified proposal rate
- reasoner calls/tokens
- peak RAM/VRAM

Do not construct one global architecture score.
Preserve the Pareto trade-off.

TESTS

Unit:

- every Pydantic contract
- ENU/NED conversion
- DecisionRouter dispatch
- stale-decision rejection
- fake-clock scheduler timing
- memory isolation/reset
- FeatureCache keys
- illegal config rejection

Contract:

- every EnvironmentAdapter -> valid ObservationPacket
- every policy -> valid DecisionEnvelope
- every ControllerAdapter consumes canonical ControlCommand
- all events contain simulation and wall-clock timestamps

Integration:

- C1/C2/C7 run in deterministic simulator
- C5 shows independent concurrent decision and monitor traces
- C6 makes zero reasoner calls in nominal episodes and calls it after an injected trigger

- C7 and C8 use exactly the same VLA plugin
- shield blocks an injected collision action

CI:

ruff
 type checker
 pytest unit
 pytest contract
 deterministic integration tests
 validate every architecture YAML

CLI

uavlab run --arch c3.yaml --env grid.yaml --seed 7
 uavlab sweep --experiment macro_screen.yaml
 uavlab replay RUN_DIR
 uavlab analyze RUN_DIRS...
 uavlab validate-config CONFIG

MILESTONES

A:

contracts
 plugin protocols
 config grammar
 scheduler
 decision router
 fake clock
 event logger
 deterministic environment

B:

all eight sentinel architecture configs with fake/oracle policies;
 prove that architectures switch only through config

C:

PX4/Gazebo and Project AirSim adapters

D:

real LLM/VLM/VLA adapters and dataset replay

E:

shared perception, KV caching, ONNX/TensorRT, quantization,
 and other deployment optimization

ACCEPTANCE CRITERIA

1. Changing C2 -> C7 requires only changing YAML.
2. The same C2 policy graph runs on multiple EnvironmentAdapters.
3. Every executed decision exposes its source observation time.

4. C5 truly runs decision and monitor asynchronously.
5. C6 invokes semantic recovery only after configured triggers.
6. C7 and C8 isolate the safety shield with identical VLA output.
7. Illegal architecture graphs fail before launch.
8. Raw model text can never reach flight-control APIs.
9. Core tests need no ROS/CUDA/network.
10. CI passes from a clean checkout.

First generate:

- repository skeleton
- interfaces/contracts
- architecture config schema
- Mermaid diagrams
- test plan

Then implement Milestone A completely before integrating a real simulator or foundation model.

This software structure is intentionally **more neutral than any existing paper runtime**. It imports AerialClaw's useful idea of bounded capabilities and platform adapters, OnFly's separation of semantic roles and later shared-perception optimization, AirHunt's asynchronous semantic/planning distinction, PMR's reasoning-admission concept, and FLIGHT/CognitiveDrone's fast-slow hierarchy without making any of those choices mandatory. ⁵³

The resulting testbed would let the actual experiment answer the question the current literature largely leaves entangled: **not which paper's complete stack wins, but where the semantic model should sit, how much authority it should receive, when it should run, what state it should remember, and what must remain outside it for safe real-time flight**. The uploaded design-space analysis found no direct prior work, through August 16, 2026, combining this factorized single-UAV runtime space, matched cross-family substitutions, and conditional multi-objective architecture search; that remains a qualified literature-search conclusion rather than proof of absence.  

¹ ¹⁴ ²⁶ OnFly: Onboard Zero-Shot Aerial Vision-Language Navigation toward Safety and Efficiency
https://arxiv.org/abs/2603.10682?utm_source=chatgpt.com

² ⁵³ AerialClaw: An Open-Source Framework for LLM-Driven Autonomous Aerial Agents
https://arxiv.org/abs/2606.12142?utm_source=chatgpt.com

³ ¹³ ³⁰ ⁴⁵ AirHunt: Bridging VLM Semantics and Continuous Planning for Efficient Aerial Object Navigation
https://arxiv.org/abs/2601.12742?utm_source=chatgpt.com

⁴ ²⁴ TypeFly: Flying Drones with Large Language Model
https://arxiv.org/abs/2312.14950?utm_source=chatgpt.com

⁵ PIVOT: Iterative Visual Prompting Elicits Actionable Knowledge for VLMs
https://arxiv.org/abs/2402.07872?utm_source=chatgpt.com

⁶ See, Point, Fly: A Learning-Free VLM Framework for Universal Unmanned Aerial Navigation
https://arxiv.org/abs/2509.22653?utm_source=chatgpt.com

- 7 23 **FM-Planner: Foundation Model Guided Path Planning for Autonomous Drone Navigation**
https://arxiv.org/abs/2505.20783?utm_source=chatgpt.com
- 8 38 **OpenFly: A Versatile Toolchain and Large-scale Benchmark for Aerial Vision-Language Navigation**
https://arxiv.org/abs/2502.18041?utm_source=chatgpt.com
- 9 **CognitiveDrone: A VLA Model and Evaluation Benchmark for Real-Time Cognitive Task Solving and Reasoning in UAVs**
https://arxiv.org/abs/2503.01378?utm_source=chatgpt.com
- 10 28 50 **GRaD-Nav++: Vision-Language Model Enabled Visual Drone Navigation with Gaussian Radiance Fields and Differentiable Dynamics**
https://arxiv.org/abs/2506.14009?utm_source=chatgpt.com
- 11 **AerialVLA: A Vision-Language-Action Model for UAV Navigation via Minimalist End-to-End Control**
https://arxiv.org/abs/2603.14363?utm_source=chatgpt.com
- 12 43 47 51 **UAV-Flow Colosseo: A Real-World Benchmark for Flying-on-a-Word UAV Imitation Learning**
https://arxiv.org/abs/2505.15725?utm_source=chatgpt.com
- 15 25 46 49 **Selective Agentic Recovery for UAV Autonomy with a Persistent Mission Runtime**
https://arxiv.org/abs/2606.14219?utm_source=chatgpt.com
- 16 27 **Think Like a Pilot: Fine-Grained Long-Horizon UAV Navigation**
https://arxiv.org/abs/2606.06836?utm_source=chatgpt.com
- 17 48 **WorldFly: A World-Model-Based Vision-Language-Action Model for UAV Navigation**
https://arxiv.org/abs/2606.06147?utm_source=chatgpt.com
- 18 40 **GitHub - NTU-ICG/FM-Planner: This is for the paper FM-Planner · GitHub**
https://github.com/ntu-icg/fm-planner?utm_source=chatgpt.com
- 19 **ICLR Poster OpenFly: A COMPREHENSIVE PLATFORM FOR AERIAL VISION-LANGUAGE NAVIGATION**
https://iclr.cc/virtual/2026/poster/10009791?utm_source=chatgpt.com
- 20 **GitHub - XuPeng23/AeroVLA: [Accepted to ECCV 2026] Official repository for AeroVLA (formerly known as AerialVLA): A Vision-Language-Action Model for UAV Navigation via Minimalist End-to-End Control · GitHub**
https://github.com/XuPeng23/AeroVLA?utm_source=chatgpt.com
- 21 39 **GitHub - Robotics-STAR-Lab/OnFly: OnFly: Onboard Zero-Shot Aerial Vision-Language Navigation toward Safety and Efficiency · GitHub**
https://github.com/Robotics-STAR-Lab/OnFly?utm_source=chatgpt.com
- 22 **Think Like a Pilot: Fine-Grained Long-Horizon UAV Navigation**
https://buaa-colalab.github.io/FLIGHT/?utm_source=chatgpt.com
- 29 **Interfaces — ROS 2 Documentation: Jazzy documentation**
https://docs.ros.org/en/jazzy/Concepts/Basic/About-Interfaces.html?utm_source=chatgpt.com
- 31 **GitHub - SJTU-ViSYS-team/VisFly: This is a fast and versatile simulator for training vision-based flight of drones. · GitHub**
https://github.com/SJTU-ViSYS-team/VisFly?utm_source=chatgpt.com
- 32 36 **Setting up a Developer Environment (Toolchain) | PX4 Guide (main)**
https://docs.px4.io/main/en/dev_setup/dev_env?utm_source=chatgpt.com

33 GitHub - microsoft/AirSim: Open source simulator for autonomous vehicles built on Unreal Engine / Unity, from Microsoft AI & Research · GitHub

https://github.com/microsoft/airsim?utm_source=chatgpt.com

34 44 GitHub - AirVLN/AirVLN · GitHub

https://github.com/AirVLN/AirVLN?utm_source=chatgpt.com

35 PX4 ROS 2 Control Interface | PX4 Guide (main)

https://docs.px4.io/main/en/ros2/px4_ros2_control_interface?utm_source=chatgpt.com

37 GitHub - Hu-chih-yao/see-point-fly: [CoRL 2025] See, Point, Fly: A Learning-Free VLM Framework for Universal Unmanned Aerial Navigation · GitHub

https://github.com/Hu-chih-yao/see-point-fly?utm_source=chatgpt.com

41 GitHub - PX4/PX4-Autopilot: PX4 Autopilot Software · GitHub

https://github.com/PX4/PX4-Autopilot?utm_source=chatgpt.com

42 GitHub - iamaissim/ProjectAirSim: Project AirSim is Microsoft's evolution of AirSim, an advanced simulation platform for building, training, and testing autonomous systems in high-fidelity virtual environments · GitHub

https://github.com/iamaissim/ProjectAirSim?utm_source=chatgpt.com

52 AutoPilot: Automating SoC Design Space Exploration for SWaP Constrained Autonomous UAVs

https://arxiv.org/abs/2102.02988?utm_source=chatgpt.com